

Algoritmos Avançados

Aulas 04 & 05: Dicionários & Tuplas em Python

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário

- 1 O TAD Mapeamento e Tabelas Hash
- 2 Dicionários em Python: Criação e Manipulação
- 3 Dicionários Avançados e Dictionary Comprehensions
- 4 O TAD Tupla em Python: Imutabilidade e Estrutura
- 5 Operações Avançadas com Tuplas
- 6 Exercícios Práticos Resolvidos

O TAD Mapeamento (Map / Dicionário)

Definition (TAD Mapeamento)

Uma coleção de pares ordenados do tipo **(chave, valor)**, denotada por (k, v) , onde cada chave k é única e serve como índice de acesso direto ao seu respectivo valor associado v .

Interface do TAD Map

- `put(k, v)`: Associa o valor v à chave k .
- `get(k)`: Recupera o valor mapeado por k .
- `remove(k)`: Remove o par associado a k .
- `containsKey(k)`: Verifica existência de k .

Aplicações no Agronegócio

- CPF/CNPJ → Registro do Produtor
- ID do Talhão → Histórico de Safra
- Código de Sensor → Telemetria (IoT)
- Vértice de Grafo → Lista de Vizinhos

Como Funcionam Tabelas Hash (Hash Tables)?

Por baixo dos panos, o dicionário em Python é implementado através de uma Tabela Hash.

O Processo de Hashing

- 1 A chave k é passada para uma função hash: $h = \text{hash}(k)$.
- 2 O valor h é mapeado para um índice no vetor interno de posições (*buckets*):

$$\text{índice} = h \pmod{N} \quad (N = \text{tamanho da tabela})$$

- 3 O par (k, v) é armazenado ou consultado diretamente naquele endereço.

Propriedade Fundamental

Permite busca, inserção e remoção em tempo médio constante: $O(1)$.

Hashability: O que pode ser Chave de Dicionário?

Para que um objeto seja utilizado como chave, ele deve ser hashable (imutável).

Tipos Hashable (Permitidos)

- int, float, bool
- str
- tuple (se contiver apenas itens imutáveis)
- frozenset

Tipos Unhashable (Proibidos)

- list (mutável)
- dict (mutável)
- set (mutável)

```
1 print(hash("talhao_01"))      # Retorna inteiro unico
2 # print(hash([1, 2, 3]))      # TypeError: unhashable type: 'list'
```

Colisão de Hash

Ocorre quando duas chaves distintas $k_1 \neq k_2$ produzem o mesmo índice na tabela ($\text{índice}(k_1) = \text{índice}(k_2)$).

- O Python resolve colisões via Endereçamento Aberto (*Open Addressing*) com sondagem pseudo-aleatória.
- Redimensionamento automático quando a taxa de ocupação (*load factor*) atinge $\approx 66\%$.

Operação	Caso Médio	Pior Caso (Muitas Colisões)
Inserção ($d[k] = v$)	$O(1)$	$O(n)$
Busca ($d[k]$, get)	$O(1)$	$O(n)$
Remoção (del , pop)	$O(1)$	$O(n)$
Verificação ($k \text{ in } d$)	$O(1)$	$O(n)$

Criando e Inicializando Dicionários

```
1 # 1. Sintaxe literal com chaves {}
2 talhao = {
3     "codigo": "T-101",
4     "cultura": "Soja",
5     "area_ha": 150.5,
6     "irrigado": True
7 }
8
9 # 2. Construtor dict() com argumentos nomeados ou lista de pares
10 propriedade = dict(nome="Fazenda Santa Rita", municipio="Bebedouro")
11 sensores = dict([("S1", 24.5), ("S2", 28.0), ("S3", 21.2)])
12
13 # 3. Metodo fromkeys() - chaves com mesmo valor padrao inicial
14 status_sensores = dict.fromkeys(["S1", "S2", "S3"], "OPERACIONAL")
15 print(status_sensores) # {'S1': 'OPERACIONAL', 'S2': 'OPERACIONAL', ...}
```

Acesso a Elementos: Colchetes vs. Método get()

```
1 safra = {"cultura": "Milho", "ano": 2026, "produtividade": 82.5}
2
3 # 1. Acesso direto com colchetes: lanca KeyError se a chave nao existir!
4 print(safra["cultura"])          # Milho
5 # print(safra["irrigacao"])      # KeyError: 'irrigacao'
6
7 # 2. Acesso seguro com get(chave, valor_padrao):
8 # Retorna None se nao encontrar (ou o valor_padrao especificado)
9 irrigado = safra.get("irrigacao")          # Retorna None
10 preco_saca = safra.get("preco_saca", 65.0) # Retorna 65.0 (padrao)
11
12 print(f"Irrigado: {irrigado} | Preço base: R$ {preco_saca:.2f}")
```

Boa Prática em Algoritmos

Prefira `d.get(k, padrao)` sempre que a chave puder estar ausente, evitando paradas abruptas do programa por exceções não tratadas.

Inserção, Atualização e.setdefault()

```
1 maquinario = {"trator": 4, "colheitadeira": 2}
2
3 # Insercao e Modificacao direta
4 maquinario["pulverizador"] = 1      # Insere nova chave
5 maquinario["trator"] = 5             # Atualiza valor existente
6
7 # Metodo setdefault(k, default)
8 # Se a chave existir, retorna seu valor atual.
9 # Se NAO existir, insere a chave com o valor default e o retorna.
10 val_tratores = maquinario.setdefault("trator", 10)      # Retorna 5 (nao
    altera)
11 val_drones = maquinario.setdefault("drones", 3)         # Insere 'drones':
    3
12
13 print(maquinario)
14 # {'trator': 5, 'colheitadeira': 2, 'pulverizador': 1, 'drones': 3}
```

Remoção de Itens em Dicionários

```
1 telemetria = {"temp": 31.2, "umidade": 65.0, "pressao": 1013.25, "vento":  
    12.0}  
2  
3 # 1. Metodo pop(k, default) - Remove e retorna o valor  
4 temp_removida = telemetria.pop("temp")  
5 print(f"Temperatura removida: {temp_removida}")    # 31.2  
6  
7 # 2. Metodo popitem() - Remove e retorna o ULTIMO par (chave, valor)  
    inserido (LIFO)  
8 ultimo_par = telemetria.popitem()  
9 print(f"Ultimo par removido: {ultimo_par}")        # ('vento', 12.0)  
10  
11 # 3. Instrucao del d[k] - Remove sem retornar  
12 del telemetria["umidade"]  
13  
14 # 4. Metodo clear() - Esvazia o dicionario  
15 telemetria.clear()  
16 print(telemetria)    # {}
```

Visões de Dicionário: `keys()`, `values()` e `items()`

Os métodos retornam objetos de visualização dinâmica (*dictionary views*) que refletem alterações na estrutura em tempo real.

```
1 precos = {"soja": 130.0, "milho": 62.0, "cafe": 950.0}
2
3 # Chaves, Valores e Pares
4 print(precos.keys())      # dict_keys(['soja', 'milho', 'cafe'])
5 print(precos.values())    # dict_values([130.0, 62.0, 950.0])
6 print(precos.items())     # dict_items([('soja', 130.0), ('milho', 62.0),
7                             ...])
8
9 # Iteracao pythonica completa com desempacotamento:
10 for commodity, valor in precos.items():
11     print(f"Cultura: {commodity:<8} | Preço/saca: R$ {valor:>7.2f}")
```

Mesclando Dicionários: update() e Operadores | / |=

```
1 dados_base = {"cultura": "Cana-de-acucar", "ano": 2026}
2 dados_sensor = {"umidade_solo": 42.5, "ano": 2026} # Sobrescreve 'ano'
3
4 # Metodo tradicional: update() (modifica in-place)
5 dados_base.update(dados_sensor)
6 print(dados_base) # {'cultura': 'Cana...', 'ano': 2026, 'umidade_solo':
   42.5}
7
8 # Operador de Uniao | (Python 3.9+): Gera um NOVO dicionario
9 d1 = {"a": 1, "b": 2}
10 d2 = {"b": 99, "c": 3}
11 d3 = d1 | d2
12 print(d3) # {'a': 1, 'b': 99, 'c': 3}
13
14 # Operador |= (Modifica in-place)
15 d1 |= d2 # d1 agora e {'a': 1, 'b': 99, 'c': 3}
```

Dictionary Comprehensions

Sintaxe concisa para transformar e filtrar dados na criação de dicionários:

```
{ chave_expr: valor_expr for item in iteravel if condicao }
```

```
1 talhoes_ha = {"T1": 120.0, "T2": 45.0, "T3": 85.5, "T4": 30.0}
2
3 # 1. Conversao de unidades (Hectares para Acres: 1 ha ~ 2.471 acres)
4 talhoes_acres = {k: v * 2.471 for k, v in talhoes_ha.items()}
5
6 # 2. Filtragem de talhoes com area maior que 50 ha
7 grandes_talhoes = {k: v for k, v in talhoes_ha.items() if v > 50.0}
8 print(grandes_talhoes) # {'T1': 120.0, 'T3': 85.5}
9
10 # 3. Inversao de Chave-Valor (se os valores forem unicos e imutaveis!)
11 codigos = {101: "SP", 102: "MG", 103: "PR"}
12 estados = {v: k for k, v in codigos.items()} # {'SP': 101, 'MG': 102, ...}
```

O Módulo collections: defaultdict

O defaultdict inicializa automaticamente um valor padrão quando uma chave inexistente é acessada pela primeira vez, eliminando checagens manuais com if.

```
1 from collections import defaultdict
2
3 # Agrupando talhoes por cultura
4 registros = [("Soja", "T1"), ("Milho", "T2"), ("Soja", "T3"), ("Algodao", "
    T4")]
5
6 # Inicializa automaticamente uma list vazia [] para novas chaves
7 talhoes_por_cultura = defaultdict(list)
8
9 for cultura, talhao in registros:
10     talhoes_por_cultura[cultura].append(talhao)
11
12 print(dict(talhoes_por_cultura))
13 # {'Soja': ['T1', 'T3'], 'Milho': ['T2'], 'Algodao': ['T4']}
```

O Módulo collections: Counter

Especializado em contagem de frequências de forma eficiente e expressiva.

```
1 from collections import Counter
2
3 leituras_pragas = ["Lagarta", "Mosca-branca", "Lagarta", "Percevejo", "
   Lagarta"]
4
5 contagem = Counter(leituras_pragas)
6 print(contagem)
7 # Counter({'Lagarta': 3, 'Mosca-branca': 1, 'Percevejo': 1})
8
9 # Obter os N elementos mais comuns
10 print(contagem.most_common(1)) # [('Lagarta', 3)]
11
12 # Atualizar contagem com novas leituras
13 contagem.update(["Percevejo", "Broca"])
14 print(contagem["Percevejo"]) # 2
```

Representação de Grafos com Dicionários

Em Algoritmos Avançados, a representação mais comum e eficiente de grafos é a Lista de Adjacência modelada via dicionário.

```
1 # Grafo de rotas logisticas entre silos (Vertices) e custos/distancias (
    Pesos)
2 grafo_logistica = {
3     "Silo_Central": {"Fazenda_A": 15, "Fazenda_B": 22},
4     "Fazenda_A":    {"Silo_Central": 15, "Fazenda_C": 30},
5     "Fazenda_B":    {"Silo_Central": 22, "Fazenda_C": 18, "Fazenda_D": 40},
6     "Fazenda_C":    {"Fazenda_A": 30, "Fazenda_B": 18, "Fazenda_D": 12},
7     "Fazenda_D":    {"Fazenda_B": 40, "Fazenda_C": 12}
8 }
9
10 # Consultando os vizinhos e o custo de conexao em O(1)
11 vizinhos_b = grafo_logistica["Fazenda_B"]
12 print("Vizinhos da Fazenda B:", list(vizinhos_b.keys())) # ['Silo_Central',
    'Fazenda_C', 'Fazenda_D']
13 print("Distancia Fazenda B -> Fazenda C:", vizinhos_b["Fazenda_C"], "km") #
    18 km
```


O que é o TAD Tupla?

Definition (Tupla)

Uma sequência ordenada e imutável de elementos heterogêneos, representada formalmente como $(a_1, a_2, \dots, a_n)$.

Propriedades Fundamentais

- **Imutabilidade:** Não permite inserção, deleção ou modificação de itens *in-place*.
- **Ordenada:** Mantém a ordem exata de inserção e suporta indexação/fatiamento.
- **Heterogênea:** Suporta múltiplos tipos de dados no mesmo registro.

Por que usar Tuplas?

- **Integridade de Dados:** Garante que valores críticos (constantes, coordenadas) não sejam alterados.
- **Otimização de Memória:** Menor consumo de bytes e alocação mais rápida que listas.
- **Chaves de Dicionários:** Podem ser usadas como chaves em tabelas hash.

Criação e Sintaxe de Tuplas

```
1 # 1. Tupla literal com parenteses
2 ponto_gps = (-20.9489, -48.4878)
3 registro = ("T-01", "Milho", 2026, 85.4)
4
5 # 2. Parenteses sao opcionais na declaracao (o operador e a VIRGULA!)
6 coordenadas = -20.9489, -48.4878
7 print(type(coordenadas))  # <class 'tuple'>
8
9 # 3. ATENCAO: Tupla de um unico elemento exige virgula final!
10 numero = (42)             # Isso e um 'int', NAO uma tupla!
11 tupla_unitaria = (42,)     # Isso e uma TUPLA valida de tamanho 1.
12
13 # 4. Construtor tuple()
14 letras = tuple(["A", "B", "C"])
```

Indexação, Fatiamento e Imutabilidade

```
1 talhao_info = ("T-Alpha", "Soja", 150.0, "Semente_BRS")
2
3 # Indexacao e Slicing (Identico a listas e strings)
4 print(talhao_info[0])      # T-Alpha
5 print(talhao_info[-1])    # Semente_BRS
6 print(talhao_info[1:3])   # ('Soja', 150.0)
7
8 # Tentativa de Modificacao lanca TypeError!
9 # talhao_info[1] = "Milho" # TypeError: 'tuple' object does not support item
   assignment
10
11 # Metodos de consulta de tupla: count() e index()
12 dados = (10, 20, 30, 20, 40)
13 print(dados.count(20))    # 2
14 print(dados.index(30))    # 2
```

A "Pegadinha" da Imutabilidade Superficial

A tupla é imutável em relação às referências que armazena, mas se contiver um objeto mutável (ex: uma lista), o estado interno desse objeto pode ser alterado!

```
1 # Tupla contendo uma lista mutavel no indice 2
2 registro = ("Fazenda Modelo", 2026, [70.5, 82.0])
3
4 # Alterando a lista DENTRO da tupla:
5 registro[2].append(95.0)
6
7 print(registro)
8 # ('Fazenda Modelo', 2026, [70.5, 82.0, 95.0]) -> O conteudo da lista mudou!
9
10 # Tentativa de substituir a lista inteira continua proibida:
11 # registro[2] = [10.0, 20.0] # TypeError!
```

Aviso para Hashing

Uma tupla que contenha listas internas não é hashable e não pode ser chave de dicionário.

Tuplas como Chaves Compostas em Dicionários

Como as tuplas são **hashables** (quando puras), elas viabilizam matrizes esparsas, planos cartesianos e indexação geográfica.

```
1 # Mapeando leituras de sensores por coordenadas de grade (X, Y)
2 grade_solo = {
3     (0, 0): {"ph": 6.2, "umidade": 45.0},
4     (0, 1): {"ph": 5.8, "umidade": 42.0},
5     (1, 0): {"ph": 6.5, "umidade": 48.0},
6     (1, 1): {"ph": 6.0, "umidade": 46.5}
7 }
8
9 # Acesso direto pelo par de coordenadas
10 coord = (1, 0)
11 print(f"Status em {coord}: pH = {grade_solo[coord]['ph']}") # pH = 6.5
```

Desempacotamento de Tuplas (*Tuple Unpacking*)

```
1 ponto = (100, 250, 45)
2
3 # 1. Desempacotamento posicional exato
4 x, y, z = ponto
5 print(f"X: {x}, Y: {y}, Z: {z}")
6
7 # 2. Desempacotamento com operador Rest (*)
8 registro = ("Fazenda Boa Vista", "Bebedouro", "Soja", "Milho", "Cafe", "
    Ativa")
9 nome, municipio, *culturas, status = registro
10
11 print(f"Nome: {nome} | Cidade: {municipio}")
12 print(f"Culturas exploradas: {culturas}") # ['Soja', 'Milho', 'Cafe']
13 print(f"Status: {status}")
14
15 # 3. Retorno de multiplos valores em funcoes
16 def estatisticas(valores: list[float]) -> tuple[float, float, float]:
17     return min(valores), sum(valores) / len(valores), max(valores)
18
19 minimo, media, maximo = estatisticas([60.0, 75.0, 90.0])
```



Tuplas Nomeadas: namedtuple

A classe namedtuple (do módulo collections) cria registros leves e legíveis acessíveis tanto por índice quanto por nome de atributo.

```
1 from collections import namedtuple
2
3 # Definindo a estrutura do registro
4 Talhao = namedtuple("Talhao", ["id", "cultura", "area_ha", "produtividade"])
5
6 # Instanciando tuplas nomeadas
7 t1 = Talhao(id=101, cultura="Soja", area_ha=80.0, produtividade=72.5)
8 t2 = Talhao(102, "Milho", 45.0, 88.0)
9
10 # Acesso intuitivo por atributo (Legível e sem custo extra de memória de
    objeto)
11 print(f"Talhao #{t1.id} -> Cultura: {t1.cultura} | Prod: {t1.produtividade}
    sc/ha")
12
13 # Continua suportando acesso por índice e desempacotamento:
14 print(t1[1])    # Soja
```

Quadro Comparativo: Lista vs. Tupla vs. Dicionário

Critério	Lista (list)	Tupla (tuple)	Dicionário (dict)
Mutabilidade	Mutável	Imutável	Mutável
Ordenação	Preservada	Preservada	Preservada (Python 3.7+)
Indexação	Inteiro ($0 \dots n - 1$)	Inteiro ($0 \dots n - 1$)	Chave Imutável (*Hash*)
Acesso Elemento	$O(1)$ por índice	$O(1)$ por índice	$O(1)$ médio por chave
Hashable?	Não	Sim (se itens forem)	Não
Uso Ideal	Coleções dinâmicas	Registros fixos / Chaves	Mapeamento e Grafos
Consumo Memória	Maior (*over-alloc*)	Menor	Médio/Alto (*hash table*)

Exercício Prático 1: Enunciado

Problema 1: Agrupamento e Contagem de Ocorrências

Dada uma lista de tuplas contendo leituras de armadilhas agrícolas no formato (data, talhao, praga, quantidade):

- 1 Construa um dicionário agrupando a quantidade total de capturas por tipo de praga.
- 2 Identifique qual praga teve o maior número total de capturas em toda a fazenda.

Exercício Prático 1: Solução Comentada

```
1 from collections import defaultdict
2
3 registros = [
4     ("2026-08-01", "T-01", "Lagarta", 12),
5     ("2026-08-01", "T-02", "Mosca-branca", 5),
6     ("2026-08-02", "T-01", "Percevejo", 8),
7     ("2026-08-02", "T-03", "Lagarta", 15),
8     ("2026-08-03", "T-02", "Lagarta", 7),
9     ("2026-08-03", "T-01", "Percevejo", 14),
10 ]
11
12 totais_por_praga = defaultdict(int)
13 for _, _, praga, qtd in registros:
14     totais_por_praga[praga] += qtd
15
16 praga_critica = max(totais_por_praga.items(), key=lambda item: item[1])
17
18 print("Total acumulado por praga:", dict(totais_por_praga))
19 print(f"Praga critica: {praga_critica[0]} com {praga_critica[1]} capturas.")
```

Exercício Prático 2: Enunciado

Problema 2: Processamento de Dados Geoespaciais com NamedTuple

Crie uma estrutura com `namedtuple` para representar coordenadas geográficas de pontos de coleta de solo `PontoColeta(id, latitude, longitude, ph, fosforo)`.

- 1 Crie uma lista com 4 pontos de coleta.
- 2 Utilizando **Dictionary Comprehension**, gere um dicionário cuja chave seja o `id` do ponto e o valor seja um booleano indicando se o solo é ideal ($\text{pH} \geq 6.0$ e $\text{fósforo} \geq 20.0$).

Exercício Prático 2: Solução Comentada

```
1 from collections import namedtuple
2
3 PontoColeta = namedtuple("PontoColeta", ["id", "lat", "lon", "ph", "fosforo"
4 ])
5
6 pontos = [
7     PontoColeta("P1", -20.941, -48.481, 6.2, 25.0),
8     PontoColeta("P2", -20.945, -48.485, 5.5, 30.0),
9     PontoColeta("P3", -20.948, -48.489, 6.5, 18.0),
10    PontoColeta("P4", -20.950, -48.492, 6.1, 22.5),
11 ]
12
13 # Dictionary Comprehension avaliando a condicao de solo ideal
14 status_pontos = {
15     p.id: (p.ph >= 6.0 and p.fosforo >= 20.0)
16     for p in pontos
17 }
18
19 for pid, ideal in status_pontos.items():
20     print(f"Ponto {pid} -> Solo Ideal: {'SIM' if ideal else 'NAO (Requer
```



Exercício Prático 3: Enunciado

Problema 3: Grafo de Adjacência e Consulta de Rotas

Implemente uma função

$\text{obter_conexoes}(\text{grafo}, \text{vertice_origem})$ que recebe um grafo representado por dicionário de adjacência e retorna

Exercício Prático 3: Solução Comentada

```
1 grafo_safra = {
2     "Sede": {"Talhao_A": 12, "Talhao_B": 4, "Silo": 8},
3     "Talhao_A": {"Sede": 12, "Silo": 6},
4     "Talhao_B": {"Sede": 4, "Silo": 15},
5     "Silo": {"Sede": 8, "Talhao_A": 6, "Talhao_B": 15}
6 }
7
8 def obter_conexoes(grafo: dict, origem: str) -> list[tuple[str, int]]:
9     if origem not in grafo:
10         return []
11     vizinhos = grafo[origem] # Retorna {'Talhao_A': 12, 'Talhao_B': 4, ...}
12     # Converte para lista de tuplas e ordena pelo peso (segundo item)
13     lista_conexoes = list(vizinhos.items())
14     lista_conexoes.sort(key=lambda conexao: conexao[1])
15     return lista_conexoes
16
17 print("Rotas a partir da Sede (ordenadas):", obter_conexoes(grafo_safra, "
18     Sede"))
19 # [('Talhao_B', 4), ('Silo', 8), ('Talhao_A', 12)]
```

Próximos Passos

Conteúdo da Próxima Aula (Aula 06)

- **Tipos Abstratos de Dados – Pilha (Stack)**
- Disciplina LIFO (*Last In, First Out*).
- Operações canônicas: `push()`, `pop()`, `peek()`, `is_empty()`.
- Implementação com Listas e com `collections.deque`.
- Aplicações clássicas: Avaliação de expressões, balanceamento de parênteses e recursão.

Tarefa Complementar

Resolver a Lista de Exercícios 3 (Dicionários e Tuplas) no repositório da disciplina.

Bons Estudos!

Dúvidas ou Perguntas?